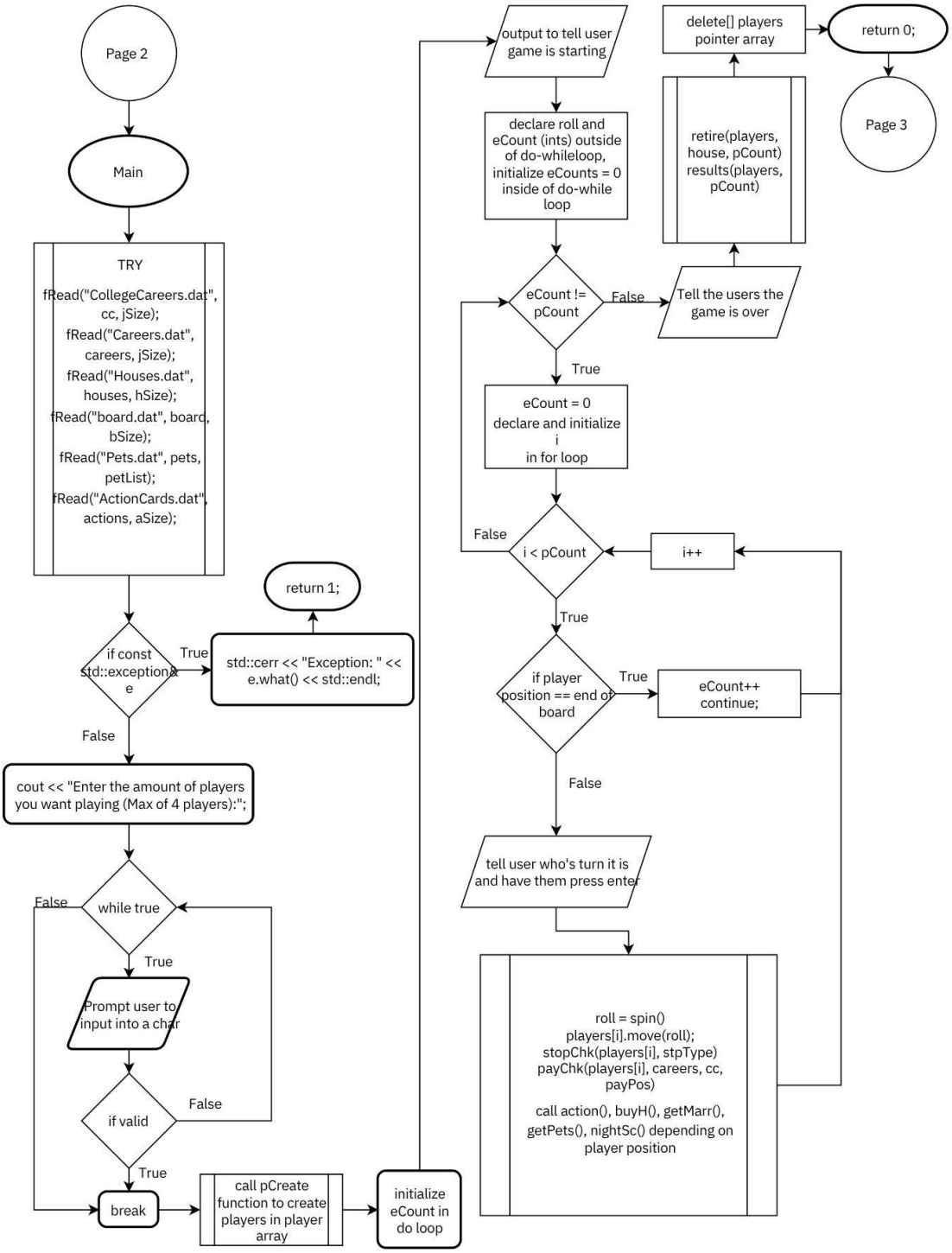
GameOfLIifeV2.cpp

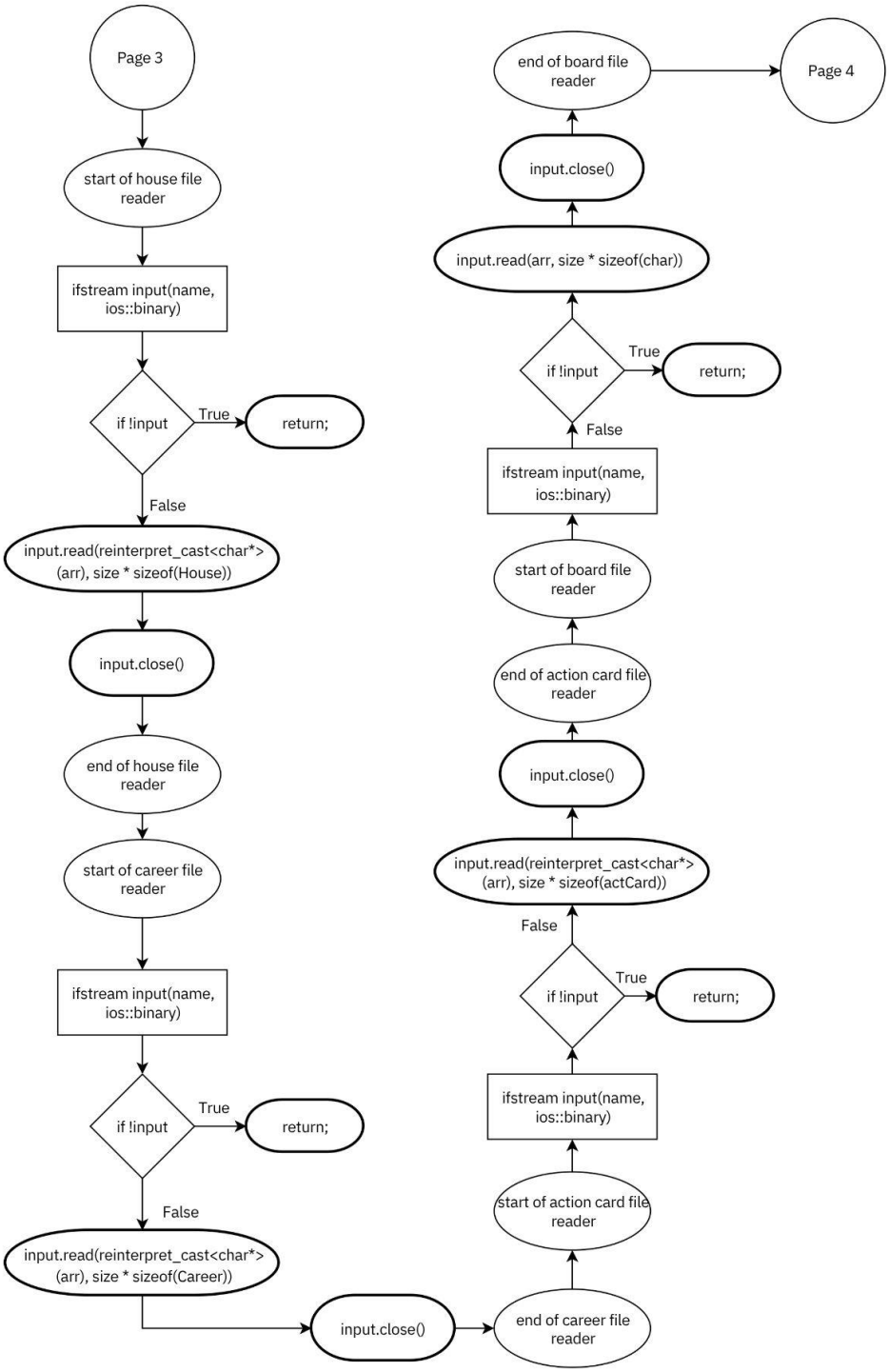
```
/*
* File: GameOfLifeV2.cpp
* Author: Keanu Arao
* Created on May 12 2025 12:01 AM
* Purpose: Recreate the board game "Life" in C++ with an emphasis on classes
* Implement "Would you like to see your cards" function
*/

//Include statements
#include <iostream>
#include <string> // Strings
#include <ctime> // Time for rand
#include <cstdlib> // Srand to set seed
#include <fstream> // Binary Files
#include <limits> // For numeric limits
// Header File inclusion
#include "Player.h"
#include "Career.h"
#include "Action.h"
#include "House.h"
#include "GameStructs.h"
#include "Pet.h"

using namespace std;
```

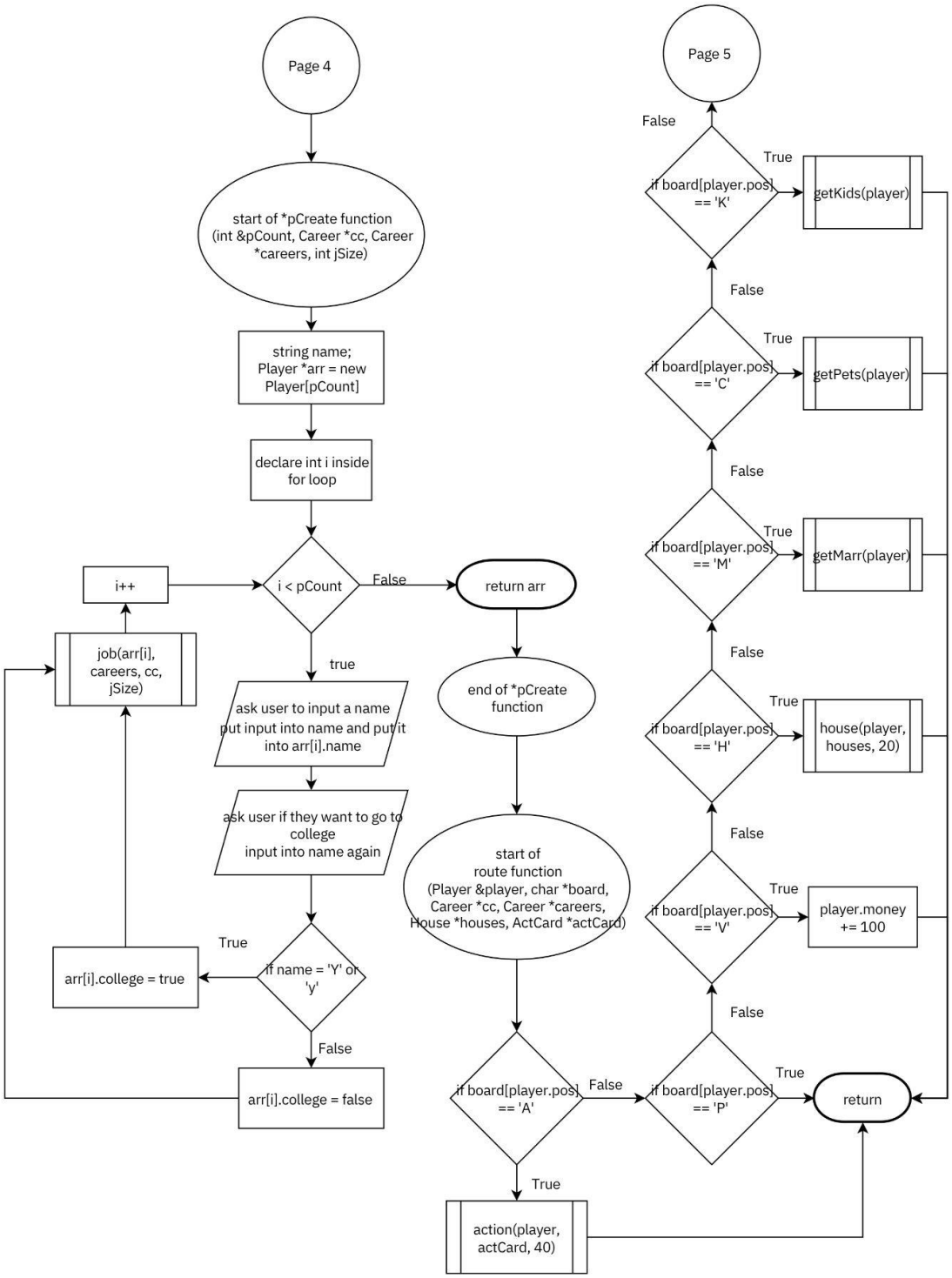
```
// File Function Prototypes
void fRead(string, Action *, int);
void fRead(string, House *, int);
void fRead(string, Career *, int);
void fRead(string, Pet *, int);
void fRead(string, char *, int);

//Board Game Function Prototypes
int spin();
void pay(Player &);
void getMarr(Player &, Player* , int);
void getKids(Player &);
void stopChk(Player &, StpType);
void payChck(Player &, PayPos);
void action(Player &, Action *, int);
void buyH(Player &, House *, int);
void sellH(Player &);
void job(Player &, Career *, Career *, int);
void jobSlct(Player &, Career *, int);
void nightSc(Player &, Career *, Career *, int);
void getPets(Player &, Pet *, int &);
void retire(Player *, House *, int);

// Program Game Function Prototypes
void winner(Player *, int);
Player *pCreate(int &, Career *, Career *, int);
void results(Player *, int);
void seeCards(Player &);
// void record -> record winner to file
// Template Function Prototypes
template <typename T>
void dispArr(const T*, int);
```

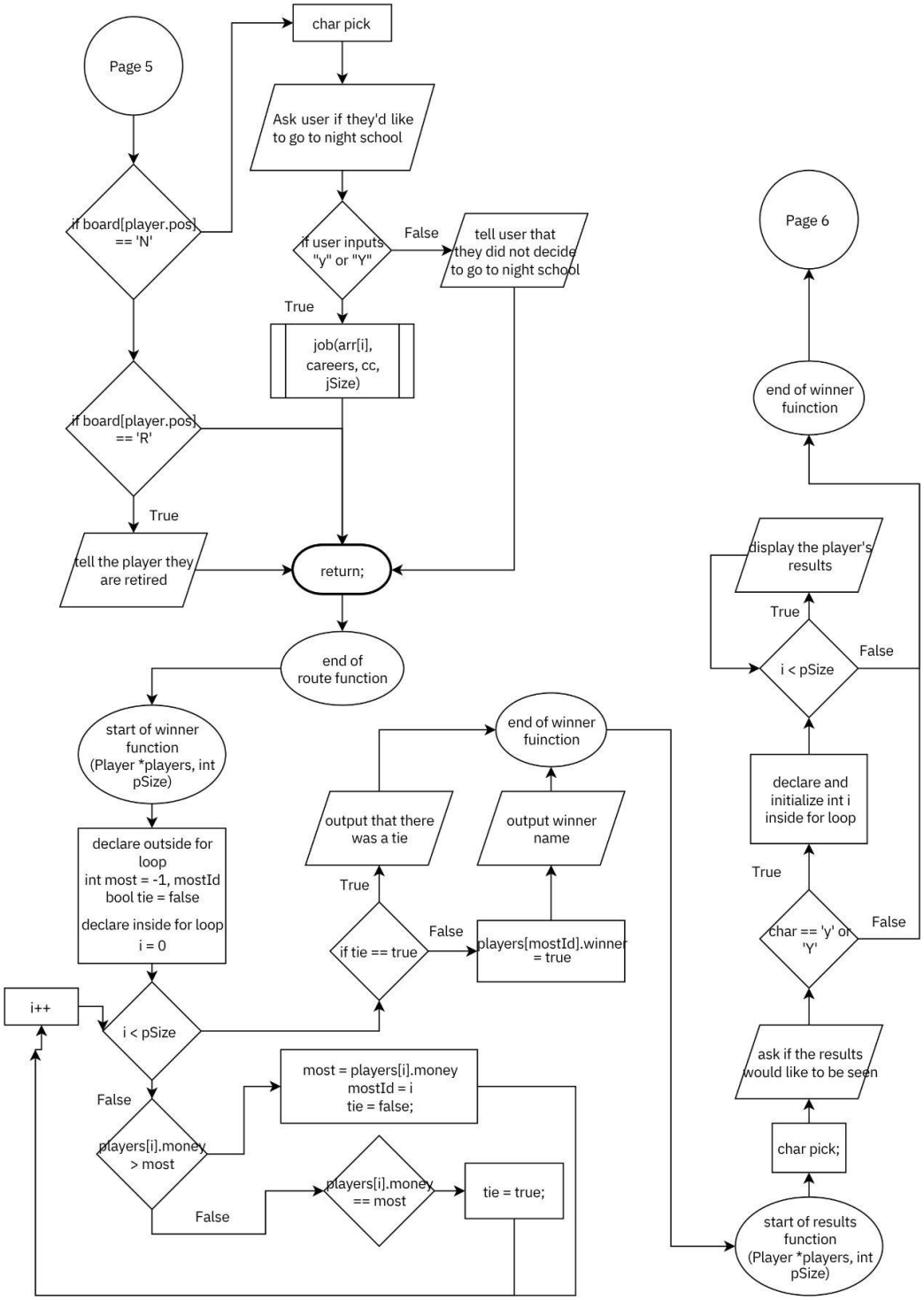
```
//// Class Implementation
void Player::setName(std::string n) {
name = n;}
int Player::getMny() const {
return money;}
```

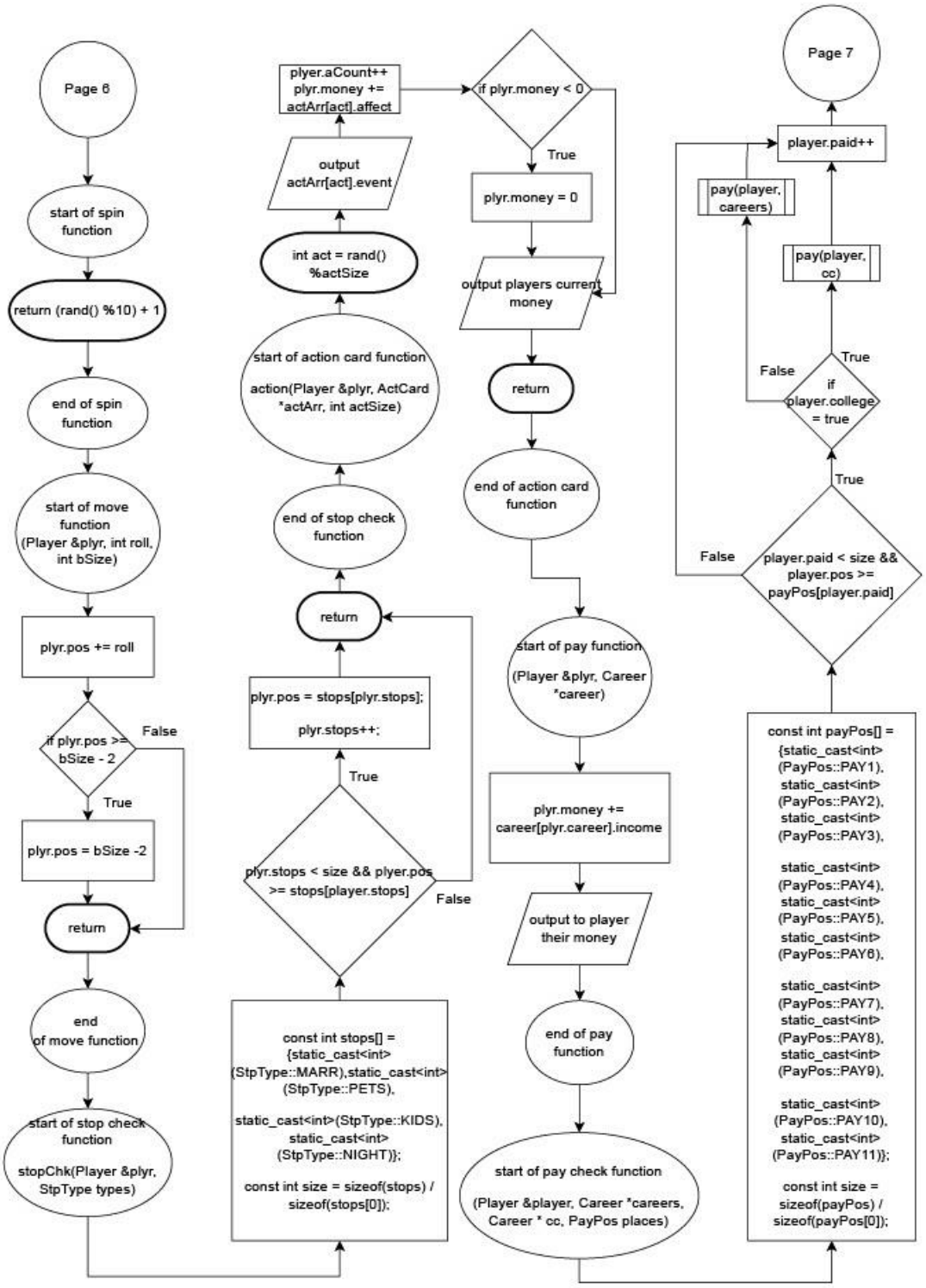
Page 2

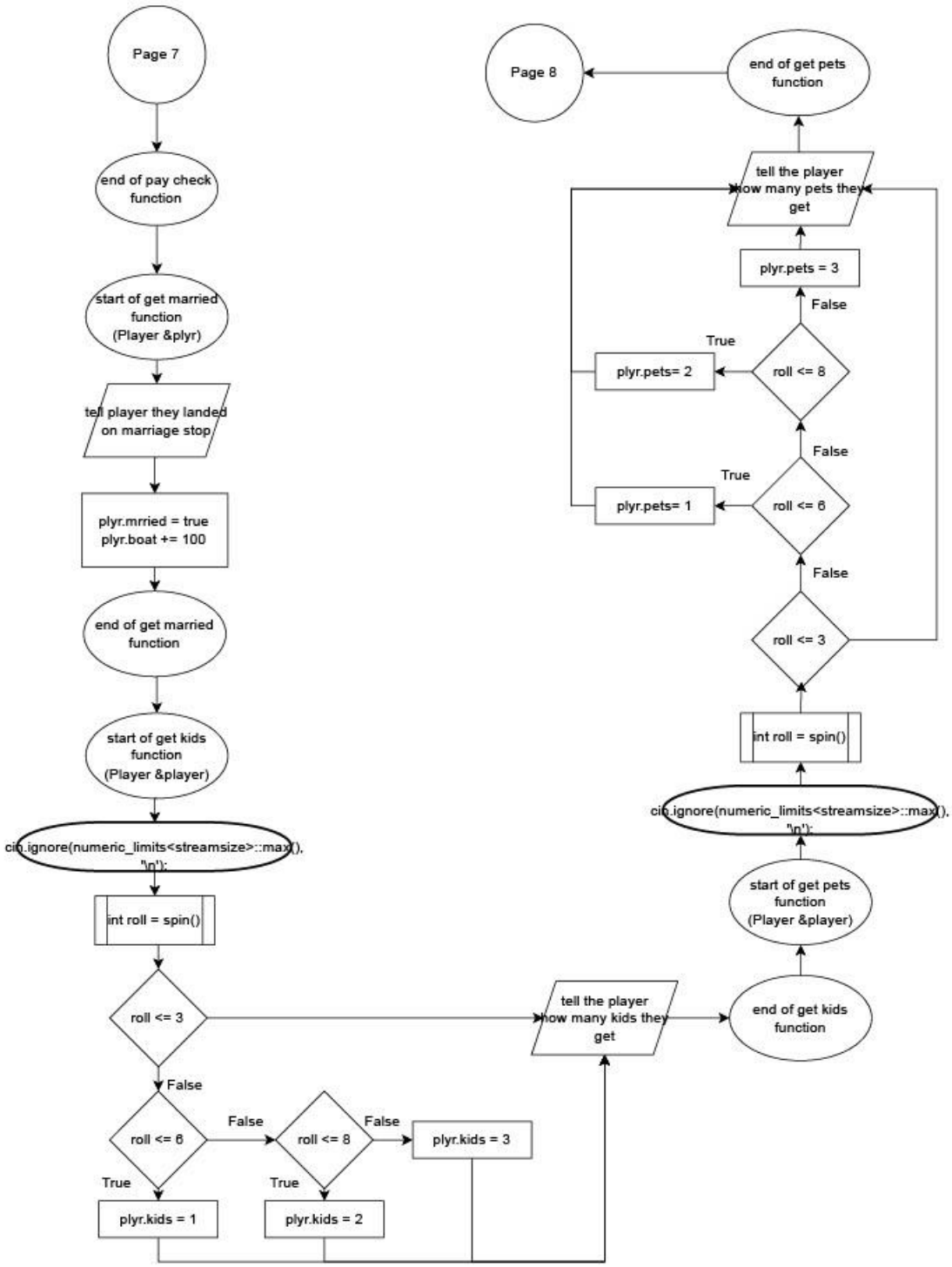

Page 2
Main
TRY
fRead("CollegeCareers.dat", cc, jSize);
fRead("Careers.dat", careers, jSize);
fRead("Houses.dat", houses, hSize);
fRead("board.dat", board, bSize);
fRead("Pets.dat", pets, petList);
fRead("ActionCards.dat", actions, aSize);
if const std::exception& e
True
std::cerr << "Exception: " << e.what() << std::endl;
return 1;
False
cout << "Enter the amount of players you want playing (Max of 4 players):";
while true
False
True
Prompt user to input into a char
if valid
False
True
break
call pCreate function to create players in player array
initialize eCount in do loop
output to tell user game is starting
declare roll and eCount (ints) outside of do-whileloop, initialize eCounts = 0 inside of do-while loop
eCount != pCount
False
Tell the users the game is over
True
eCount = 0
declare and initialize i
in for loop
i < pCount
False
i++
True
if player position == end of board
True
eCount++
continue;
False
tell user who's turn it is and have them press enter
roll = spin()
players[i].move(roll);
stopChk(players[i], stpType)
payChk(players[i], careers, cc, payPos)
call action(), buyH(), getMarr(), getPets(), nightSc() depending on player position
retire(players, house, pCount)
results(players, pCount)
delete[] players pointer array
return 0;
Page 3

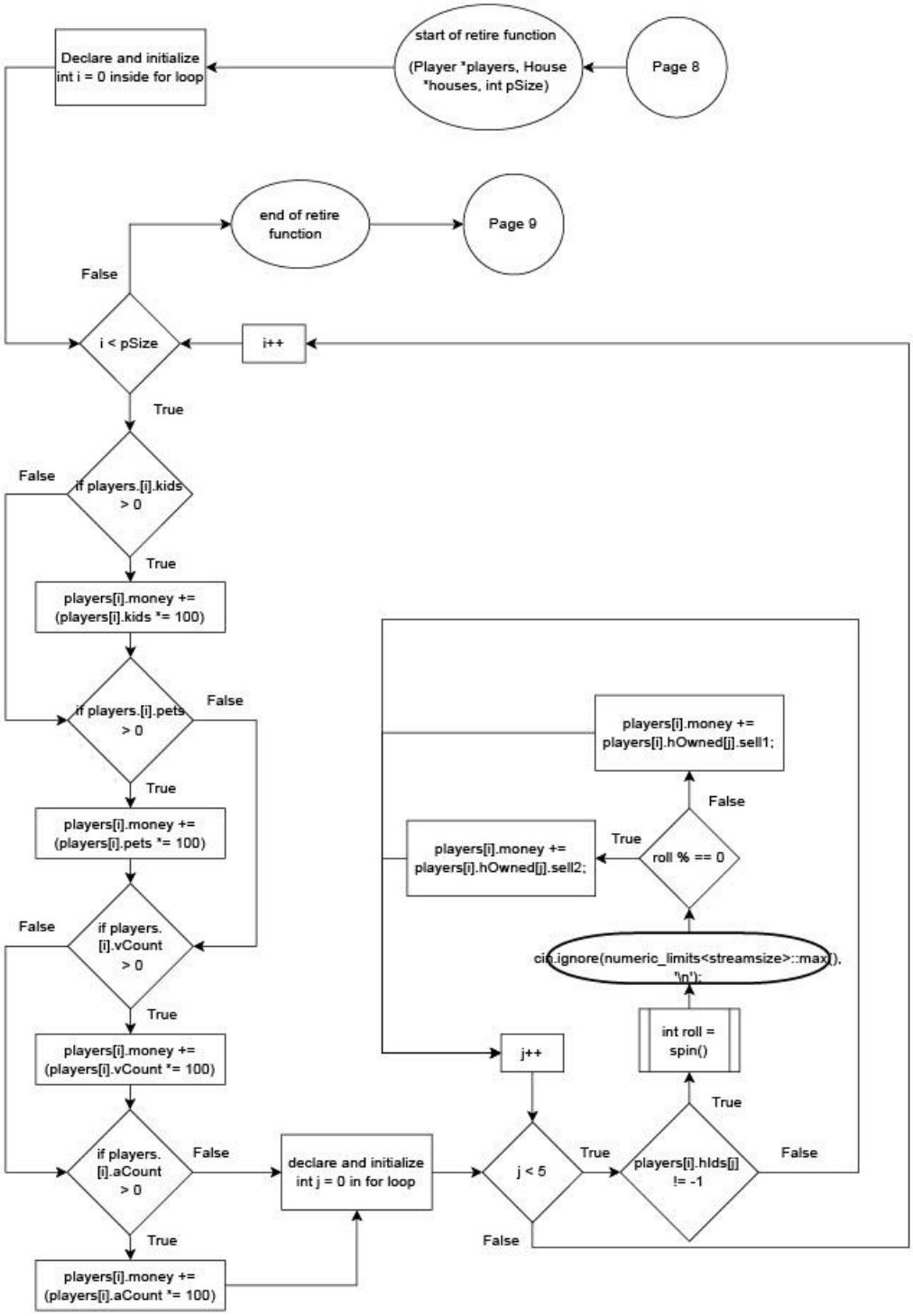

Page 3
start of house file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(House))
input.close()
end of house file reader
start of career file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(Career))
input.close()
end of career file reader
start of action card file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(actCard))
input.close()
end of action card file reader
start of board file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(arr, size * sizeof(char))
input.close()
end of board file reader
Page 4

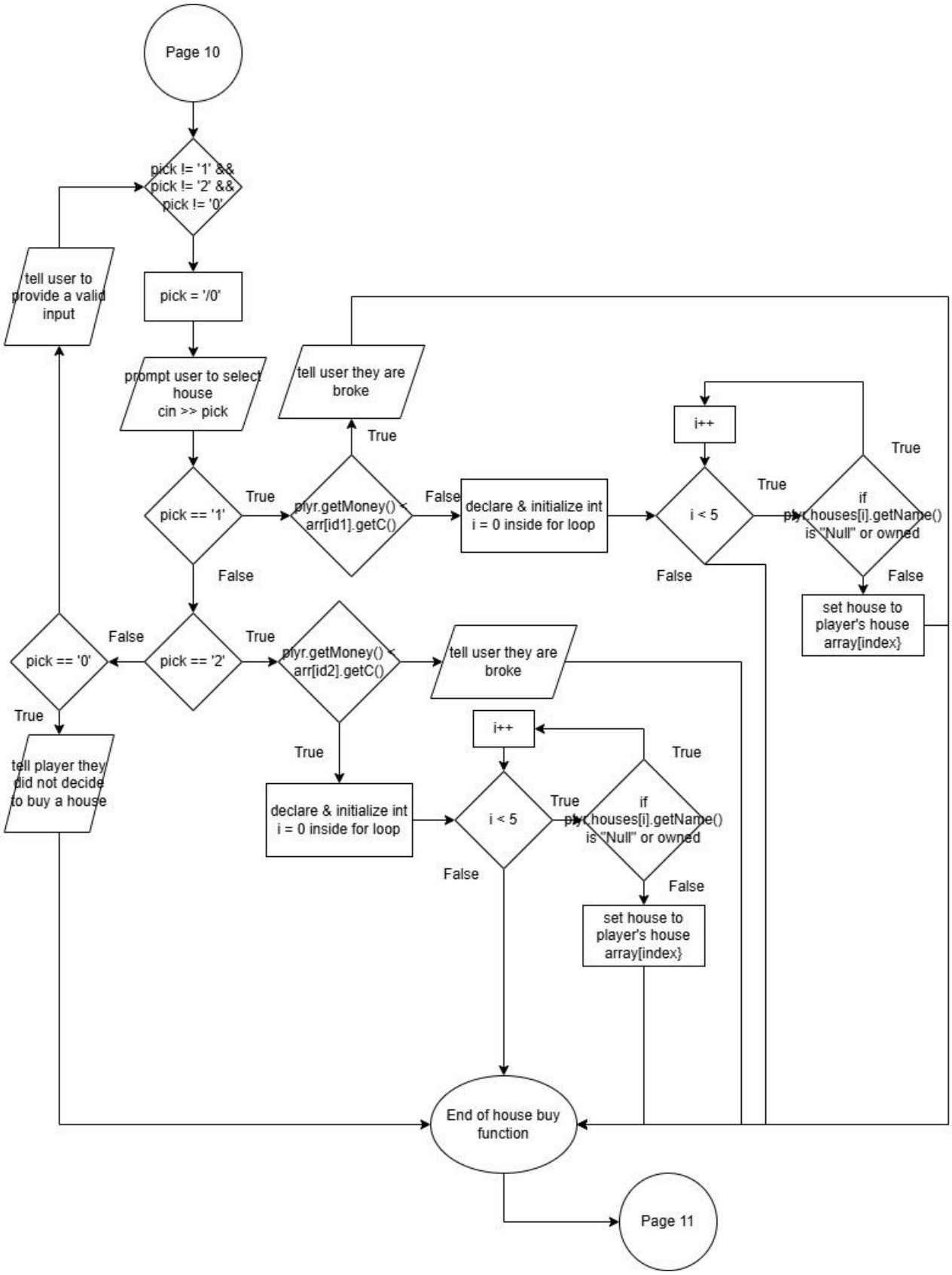

Page 4
start of *pCreate function (int &pCount, Career *cc, Career *careers, int jSize)
string name; Player *arr = new Player[pCount]
declare int i inside for loop
i < pCount
False
return arr
true
ask user to input a name put input into name and put it into arr[i].name
ask user if they want to go to college input into name again
if name = 'Y' or 'y'
True
arr[i].college = true
i++
job(arr[i], careers, cc, jSize)
False
arr[i].college = false
end of *pCreate function
start of route function (Player &player, char *board, Career *cc, Career *careers, House *houses, ActCard *actCard)
if board[player.pos] == 'A'
True
action(player, actCard, 40)
False
if board[player.pos] == 'P'
True
return
False
if board[player.pos] == 'V'
True
player.money += 100
False
if board[player.pos] == 'H'
True
house(player, houses, 20)
False
if board[player.pos] == 'M'
True
getMarr(player)
False
if board[player.pos] == 'C'
True
getPets(player)
False
if board[player.pos] == 'K'
True
getKids(player)
False
Page 5

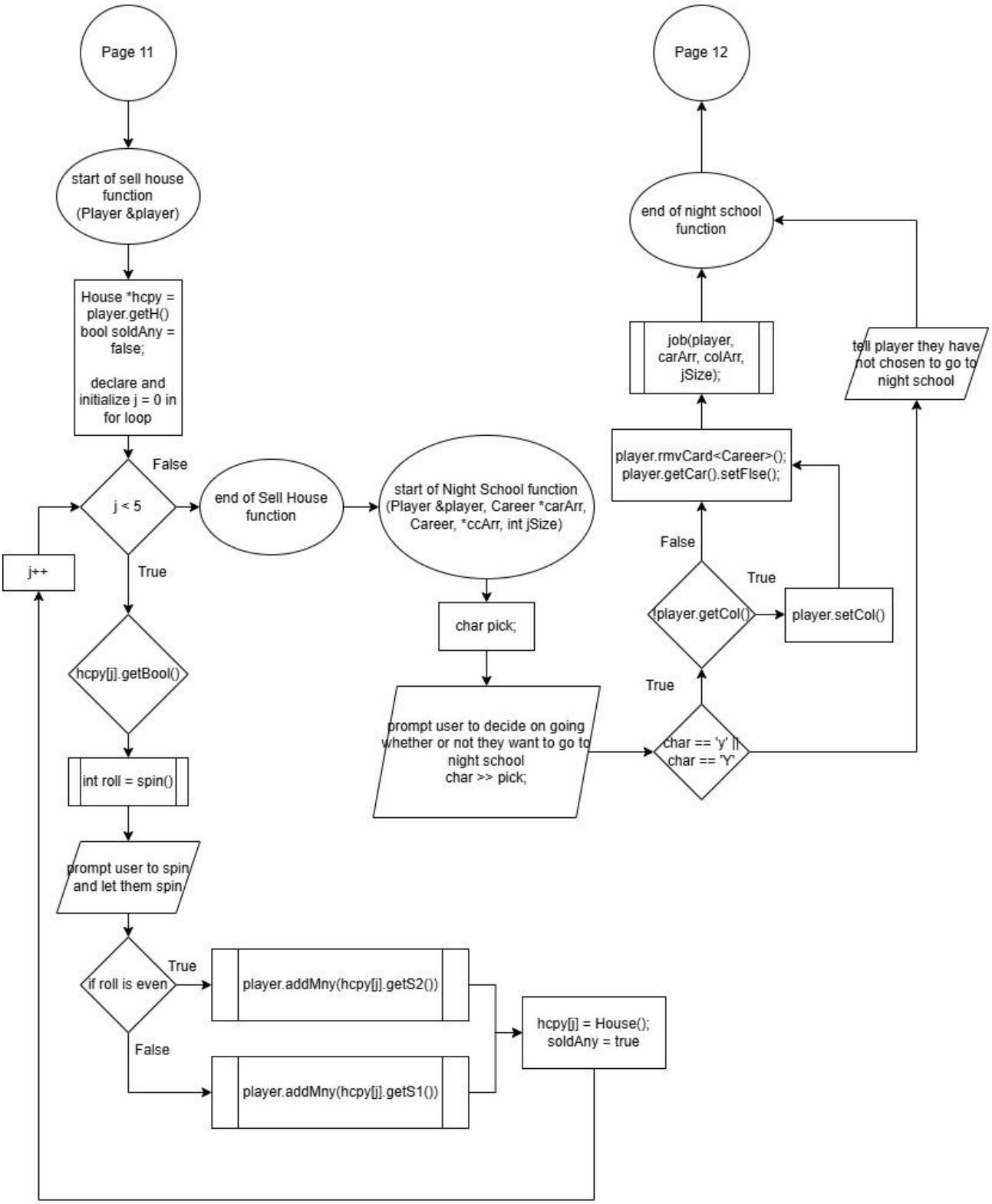

Page 5
char pick
Ask user if they'd like to go to night school
if board[player.pos] == 'N'
if user inputs "y" or "Y"
False
tell user that they did not decide to go to night school
True
job(arr[i], careers, cc, jSize)
if board[player.pos] == 'R'
True
tell the player they are retired
return;
end of route function
start of winner function (Player *players, int pSize)
declare outside for loop
int most = -1, mostId
bool tie = false
declare inside for loop
i = 0
i++
i < pSize
False
players[i].money > most
most = players[i].money
mostId = i
tie = false;
False
players[i].money == most
tie = true;
if tie == true
True
output that there was a tie
False
players[mostId].winner = true
output winner name
end of winner fuinction
start of results function (Player *players, int pSize)
char pick;
ask if the results would like to be seen
char == 'y' or 'Y'
True
False
declare and initialize int i inside for loop
i < pSize
True
False
display the player's results
end of winner fuinction
Page 6

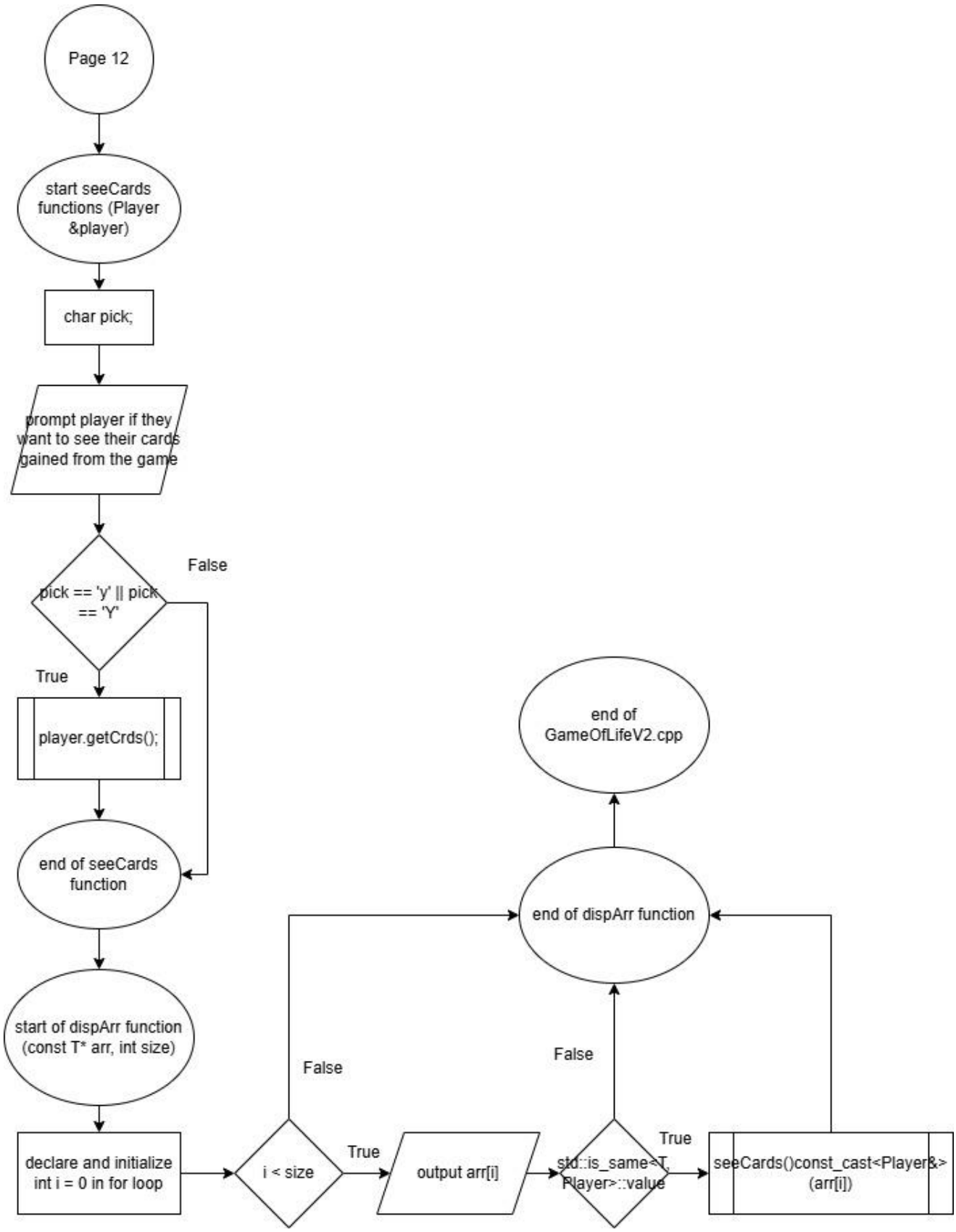

Page 6
start of spin function
return (rand() %10) + 1
end of spin function
start of move function (Player &plyr, int roll, int bSize)
plyr.pos += roll
if plyr.pos >= bSize - 2
False
True
plyr.pos = bSize -2
return
end of move function
start of stop check function
stopChk(Player &plyr, StpType types)
const int stops[] = {static_cast<int> (StpType::MARR),static_cast<int> (StpType::PETS),
static_cast<int>(StpType::KIDS), static_cast<int> (StpType::NIGHT)};
const int size = sizeof(stops) / sizeof(stops[0]);
plyr.stops < size && plyer.pos >= stops[player.stops]
True
False
plyr.pos = stops[plyr.stops];
plyr.stops++;
return
end of stop check function
start of action card function
action(Player &plyr, ActCard *actArr, int actSize)
int act = rand() %actSize
output actArr[act].event
plyer.aCount++ plyr.money += actArr[act].affect
if plyr.money < 0
True
plyr.money = 0
output players current money
return
end of action card function
start of pay function
(Player &plyr, Career *career)
plyr.money += career[plyr.career].income
output to player their money
end of pay function
start of pay check function
(Player &player, Career *careers, Career * cc, PayPos places)
const int payPos[] = {static_cast<int> (PayPos::PAY1), static_cast<int> (PayPos::PAY2), static_cast<int> (PayPos::PAY3),
static_cast<int> (PayPos::PAY4), static_cast<int> (PayPos::PAY5), static_cast<int> (PayPos::PAY6),
static_cast<int> (PayPos::PAY7), static_cast<int> (PayPos::PAY8), static_cast<int> (PayPos::PAY9),
static_cast<int> (PayPos::PAY10), static_cast<int> (PayPos::PAY11)};
const int size = sizeof(payPos) / sizeof(payPos[0]);
player.paid < size && player.pos >= payPos[player.paid]
True
False
if player.college = true
True
False
pay(player, cc)
pay(player, careers)
player.paid++
Page 7


Page 7
end of pay check function
start of get married function (Player &plyr)
tell player they landed on marriage stop
plyr.mrried = true
plyr.boat += 100
end of get married function
start of get kids function (Player &player)
cin.ignore(numeric_limits<streamsize>::max(), '\n');
int roll = spin()
roll <= 3
False
roll <= 6
True
plyr.kids = 1
False
roll <= 8
True
plyr.kids = 2
False
plyr.kids = 3
tell the player how many kids they get
end of get kids function
start of get pets function (Player &player)
cin.ignore(numeric_limits<streamsize>::max(), '\n');
int roll = spin()
roll <= 3
False
roll <= 6
True
plyr.pets= 1
False
roll <= 8
True
plyr.pets= 2
False
plyr.pets = 3
tell the player how many pets they get
end of get pets function
Page 8


start of retire function
(Player *players, House *houses, int pSize)
Page 8
Declare and initialize int i = 0 inside for loop
end of retire function
Page 9
False
i < pSize
i++
True
False
if players.[i].kids > 0
True
players[i].money += (players[i].kids *= 100)
if players.[i].pets > 0
False
True
players[i].money += (players[i].pets *= 100)
False
if players. [i].vCount > 0
True
players[i].money += (players[i].vCount *= 100)
if players. [i].aCount > 0
False
True
players[i].money += (players[i].aCount *= 100)
declare and initialize int j = 0 in for loop
j < 5
True
False
players[i].hIds[j] != -1
True
False
int roll = spin()
cin.ignore(numeric_limits<streamsize>::max(), '\n');
roll % == 0
True
players[i].money += players[i].hOwned[j].sell2;
False
players[i].money += players[i].hOwned[j].sell1;
j++


Page 10
pick != '1' &&
pick != '2' &&
pick != '0'
tell user to provide a valid input
pick = '/0'
prompt user to select house
cin >> pick
tell user they are broke
True
pick == '1'
True
plyr.getMoney() < arr[id1].getC()
False
declare & initialize int i = 0 inside for loop
i++
True
True
i < 5
if plyr.houses[i].getName() is "Null" or owned
False
False
set house to player's house array[index}
False
pick == '0'
False
pick == '2'
True
plyr.getMoney() < arr[id2].getC()
tell user they are broke
True
tell player they did not decide to buy a house
True
i++
True
declare & initialize int i = 0 inside for loop
i < 5
True
if plyr.houses[i].getName() is "Null" or owned
False
False
set house to player's house array[index}
End of house buy function
Page 11


Page 11
start of sell house function (Player &player)
House *hcpy = player.getH() bool soldAny = false;
declare and initialize j = 0 in for loop
j < 5
False
True
end of Sell House function
start of Night School function (Player &player, Career *carArr, Career, *ccArr, int jSize)
j++
hcpy[j].getBool()
int roll = spin()
prompt user to spin and let them spin
if roll is even
True
False
player.addMny(hcpy[j].getS2())
player.addMny(hcpy[j].getS1())
hcpy[j] = House(); soldAny = true
char pick;
prompt user to decide on going whether or not they want to go to night school char >> pick;
char == 'y' || char == 'Y'
True
!player.getCol()
True
False
player.setCol()
player.rmvCard<Career>(); player.getCar().setFlse();
job(player, carArr, colArr, jSize);
tell player they have not chosen to go to night school
end of night school function
Page 12


Page 12
start seeCards functions (Player &player)
char pick;
prompt player if they want to see their cards gained from the game
pick == 'y' || pick == 'Y'
False
True
player.getCrds();
end of seeCards function
start of dispArr function (const T* arr, int size)
declare and initialize int i = 0 in for loop
i < size
True
False
output arr[i]
std::is_same<T, Player>::value
True
False
seeCards()const_cast<Player&> (arr[i])
end of dispArr function
end of GameOfLifeV2.cpp